Développer pour le Cloud

2025-26 Valentin PICARD
DEVFLSTK MAST2 B

Un peu d’histoire

2003 — Le problème de Google

En 2003, Google a un problème que personne d'autre n'a

- Google gère des millions de machines (Gmail, Search, Maps, YouTube..)
- Tout tourne en même temps, sur des milliers de serveurs répartis dans des datacenters à travers le monde.

Une petite équipe d’ingénieurs commence à travailler sur un système interne de gestion de containers.

- Ainsi né “Borg” (référence aux aliens de Star Trek)
- Une conscience collective qui partage les ressources.

2013 — Docker change tout

En mars 2013, Solomon Hykes présente Docker en 5 minutes à PyCon, Docker devient viral.

- Le monde entier découvre les containers.
- … Google l'utilise depuis une décennie.

Chez Google, on sait ce que Borg peut faire et ce que Docker ne peut pas faire.

- Docker tourne sur **une seule machine.**
- Borg orchestre des **dizaines de milliers de machines**.

2014 — Le premier commit

"From a business perspective, if a product is massively successful and you're the only one who can sell it, you're in a great position. But Kubernetes would only be massively successful if it had an ecosystem — and the best way to foster an ecosystem is open source."

— Brendan Burns

La vraie bataille n'est pas technique, il faut convaincre Google de rendre le projet **open source**.

- Le 6 juin 2014, le premier commit est pushé sur GitHub
- Le 10 juin, Eric Brewer annonce Kubernetes en keynote à DockerCon.

Son nom vient du grec ancien **κυβερνήτης** (*timonier*), celui qui tient la barre.

2015 — Google lâche le contrôle

En juillet 2015, Google donne Kubernetes à la **CNCF** (Cloud Native Computing Foundation), une fondation neutre rattachée à la Linux Foundation.

Google abandonne volontairement le contrôle d'un de ses projets les plus stratégiques.

Microsoft, RedHat, IBM, Docker rejoignent la communauté dès le premier jour.

2017 — La guerre est finie

Trois prétendants s'affrontent pour devenir le standard de l'orchestration :

- **Docker Swarm** — l'outil natif de Docker, simple mais limité
- **Apache Mesos** — puissant, complexe, adopté par Twitter et Airbnb
- **Kubernetes** — plus difficile à prendre en main, mais plus complet

En octobre 2017, Docker annonce qu'il intègre Kubernetes dans Docker Desktop, **Docker Swarm capitule**. Mesos est progressivement abandonné.

La guerre est finie. Kubernetes a gagné, pas par la force, mais par l'écosystème.

Kubernetes

Kubernetes

1. Maintien d’un état
2. Vue d’ensemble des composants
3. Pod et Deployment
4. Le service
5. L’ingress
6. ConfigMap
7. Secrets
8. Statefulset
9. Namespaces
10. kubectl

1 - Maintien d’un état désiré

	Demande	Réaction
Docker	“Lance le conteneur `task-service`”	Lance le conteneur et ça s’arrête là
k8s	"Je veux 2 replicas de `task-service`"	- créé - détruit - redémarre des conteneurs pour maintenir cet état en permanence.

1 - Maintien d’un état désiré

YAML (état désiré)

↕ Kubernetes compare en permanence

Cluster (état réel)

→ Écart détecté → Kubernetes corrige automatiquement

2 - Vue d'ensemble des composants

```
Pod              ← unité minimale : 1 conteneur (+ éventuellement un sidecar)
  ↑
Deployment       ← maintient N Pods, gère les updates et rollbacks
  ↑
Service          ← adresse stable vers les Pods
  ↑
Ingress          ← point d'entrée depuis l'extérieur

+ ConfigMap      ← configuration non sensible
+ Secret         ← mots de passe, tokens
+ StatefulSet    ← comme Deployment, mais pour les workloads avec état (DB)
+ Namespace      ← isolation logique (local, staging, production...)
```

3 - Le pod

- Un Pod = 1 ou plusieurs conteneurs qui partagent le même réseau et les mêmes volumes.

En pratique : **1 Pod = 1 conteneur** dans la très grande majorité des cas.

Le cas à plusieurs conteneurs s'appelle le pattern **sidecar** :

- un conteneur secondaire qui assiste le principal (agent de logs, proxy...). C'est un cas avancé.

3 - Deployment et ReplicaSet

- C’est le `deployment.yaml` qui s'occupe des Pods

La chaîne :

- Deployment → crée automatiquement un ReplicaSet → qui crée les Pods

Le ReplicaSet :

- composant Kubernetes créé automatiquement par le Deployment.
- replicas: 2 est l'instruction donnée au Deployment, qui la transmet au ReplicaSet.

```yaml
# deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: task-service
  namespace: local            # ou staging, production selon l'env
spec:
  replicas: 2
  selector:
    matchLabels:
      app: task-service
  template:
    metadata:
      labels:
        app: task-service
    spec:
      containers:
        - name: task-service
          image: <user>/taskflow-task-service:latest
          ports:
            - containerPort: 3002
```

3 - Liveness et Readiness probes

- l'endpoint `/health` reste le même
- la différence est dans la réaction de Kubernetes

Probe	Échec	Ce qu'on voit
Readiness	Pod retiré du Service, plus de trafic, conteneur toujours vivant	READY: 0/1
Liveness	Conteneur tué et redémarré automatiquement	colonne RESTARTS augmente

```yaml
# deployment.yaml — dans la section containers
containers:
  - name: task-service
    image: <user>/taskflow-task-service:latest
    readinessProbe:     # "est-ce que ce Pod peut recevoir du trafic ?"
      httpGet:
        path: /health
        port: 3002
      initialDelaySeconds: 10
      periodSeconds: 5
    livenessProbe:      # "est-ce que ce Pod est encore vivant ?"
      httpGet:
        path: /health
        port: 3002
      initialDelaySeconds: 15
      periodSeconds: 10
```

Vue d'ensemble des composants

```
Pod              ← unité minimale : 1 conteneur (+ éventuellement un sidecar)
  ↑
Deployment       ← maintient N Pods, gère les updates et rollbacks
```

4 - Le service

Le problème : **Les Pods ont des IPs dynamiques**.

À chaque création, l'IP change. Comment les autres services savent où envoyer le trafic ?

4 - Le service

Le problème : **Les Pods ont des IPs dynamiques**.

À chaque création, l'IP change. Comment les autres services savent où envoyer le trafic ?

Le Service connaît toujours les Pods actifs grâce aux labels.

```
user-service → http://task-service:3002
                        ↓
                Service (IP stable)
              ↙         ↓         ↘
           Pod A     Pod B     Pod C
```

```yaml
# service.yaml
apiVersion: v1
kind: Service
metadata:
  name: task-service
  namespace: local
spec:
  selector:
    app: task-service      # envoie vers tous les Pods avec ce label
  ports:
    - port: 3002
      targetPort: 3002
```

Vue d'ensemble des composants

```
Pod            ← unité minimale : 1 conteneur (+ éventuellement un sidecar)
  ↑
Deployment     ← maintient N Pods, gère les updates et rollbacks
  ↑
Service        ← adresse stable vers les Pods
```

5 - L’ingress

L'Ingress = un fichier YAML de règles de routage HTTP

Ce fichier seul ne fait rien.

Il faut l'Ingress Controller qui lit ces règles et fait le routage.

Pour tester sans Ingress en local :

```
kubectl port-forward svc/task-service 3002:3002 -n dev
```

Tunnel temporaire entre ta machine et le Service, le temps que la commande tourne.

Pratique pour déboguer, pas une architecture réelle.

```yaml
# ingress.yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: taskflow-ingress
  namespace: local
spec:
  rules:
    - http:
        paths:
          - path: /api/tasks
            pathType: Prefix
            backend:
              service:
                name: task-service
                port:
                  number: 3002
          - path: /api/users
            pathType: Prefix
            backend:
              service:
                name: user-service
                port:
                  number: 3001
```

Vue d'ensemble des composants

```
Pod          ← unité minimale : 1 conteneur (+ éventuellement un sidecar)
  ↑
Deployment   ← maintient N Pods, gère les updates et rollbacks
  ↑
Service      ← adresse stable vers les Pods
  ↑
Ingress      ← point d'entrée depuis l'extérieur
```

6 - ConfigMap

Principe :

- La configuration change entre les environnements. Le code non.
- On ne met jamais les variables en dur dans l'image Docker.

```yaml
# configmap.yaml — exemple pour local
apiVersion: v1
kind: ConfigMap
metadata:
  name: task-service-config
  namespace: local              # staging ou production
data:
  PORT: "3002"
  DATABASE_URL: postgresql://taskflow:taskflow@postgres:5432/taskflow
  LOG_LEVEL: debug              # debug en local, warn en production
  OTEL_SERVICE_NAME: task-service

---

# deployment.yaml
containers:
  - name: task-service
    envFrom:
      - configMapRef:
          name: task-service-config
```

7 - Secrets

Il existe deux types de secrets dans un projet Kubernetes.

	Secrets applicatifs	Secrets de pipeline
Exemples	`JWT_SECRET`, `DB_PASSWORD`	`DOCKERHUB_TOKEN`, kubeconfig
Où ils vivent	Dans le cluster Kubernetes	Dans GitHub Secrets
Qui les utilise	Les Pods au démarrage	GitHub Actions uniquement
Format	Objet Kubernetes (`Secret`)	Variable d'environnement CI
Entrent dans le cluster	Oui	Non

```yaml
# secret.yaml
apiVersion: v1
kind: Secret
metadata:
  name: task-service-secrets
  namespace: local
type: Opaque
stringData:          # on écrit en clair, Kubernetes encode en base64
  JWT_SECRET: super-secret
  DB_PASSWORD: taskflow

---

# deployment.yaml
containers:
  - name: task-service
    envFrom:
      - configMapRef:
          name: task-service-config
      - secretRef:
         # Kubernetes injecte les valeurs au démarrage du Pod
          name: task-service-secrets
```

Quelle mesure doit-on prendre avec ce fichier `secret.yaml` ?

7 - Secrets applicatifs

(secrets injectés dans les Pods)

⚠️ **Règle absolue :**

- Ne jamais commiter un `secret.yaml` avec de vraies valeurs dans Git.
- On l'applique manuellement :
  `kubectl apply -f secret.yaml -n local`
- On le garde localement, pas dans le repo.

7 - Secrets de pipeline

Dans GitHub Settings → Secrets → Actions :

- `DOCKERHUB_USERNAME` ← utilisé par la CI pour pusher les images
- `DOCKERHUB_TOKEN` ← idem

Ces secrets n'entrent jamais dans le cluster, ils sont utilisés uniquement par le workflow GitHub Actions.

Mais le cluster, lui, doit pouvoir puller les images Docker Hub.

7 - Secrets de pipeline

On crée manuellement un Secret `docker-registry` dans le cluster :

```
kubectl create secret docker-registry dockerhub-credentials \
  --docker-username=<votre-username> \
  --docker-password=<votre-token> \
  --docker-server=https://index.docker.io/v1/ \
  --namespace local
```

```yaml
# dockerhub-credentials.yaml créé par la commande
apiVersion: v1
data:
  .dockerconfigjson: <token>
kind: Secret
metadata:
  name: dockerhub-credentials
  namespace: staging
type: kubernetes.io/dockerconfigjson

---

# deployment.yaml
spec:
  imagePullSecrets:
    - name: dockerhub-credentials
     # Kubernetes utilise ce Secret pour puller l'image
  containers:
    - name: task-service
      image: monuser/taskflow-task-service:latest
```

Vue d'ensemble des composants

```
Pod              ← unité minimale : 1 conteneur (+ éventuellement un sidecar)
  ↑
Deployment       ← maintient N Pods, gère les updates et rollbacks
  ↑
Service          ← adresse stable vers les Pods
  ↑
Ingress          ← point d'entrée depuis l'extérieur

+ ConfigMap      ← configuration non sensible
+ Secret         ← mots de passe, tokens
```

8 - StatesulSet

Pourquoi pas un Deployment pour un service stateful (eg. postgres) ?

- Un Deployment crée des Pods interchangeables.
- Postgres a besoin de retrouver ses données au re-démarrage
- Il lui faut un volume attaché à son identité.

	Deployment	**StatefulSet**
Pods	Interchangeables	Identité stable (db-0, db-1...)
Stockage	Éphémère	Volume persistant par replica
Démarrage	Aléatoire	Ordonné

```yaml
# statefulset.yaml (1/2)
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: postgres
  namespace: local
spec:
  serviceName: postgres
  replicas: 1
  selector:
    matchLabels:
      app: postgres
  template:
    metadata:
      labels:
        app: postgres
         #[...]

# Peu de changement vis à vis d'un deployment
```

```yaml
# statefulset.yaml (2/2)
spec:
  # [...]
  template:
    # [...]
    spec:
      containers:
        - name: postgres
          image: postgres:16-alpine
          envFrom:
            - secretRef:
                name: postgres-secret
          volumeMounts:
            - name: postgres-data
              mountPath: /var/lib/postgresql/data
  volumeClaimTemplates:
    - metadata:
        name: postgres-data
      spec:
        accessModes: ["ReadWriteOnce"]
        resources:
          requests:
            storage: 1Gi
```

8 - StatesulSet

`volumeClaimTemplates`

- Pour chaque réplica, Kubernetes crée un volume persistant dédié.
- Si `postgres-0` redémarre, il retrouve exactement son volume `postgres-data-postgres-0`

`ReadWriteOnce`

- Postgres ne doit jamais avoir deux processus qui écrivent simultanément sur le même stockage.

Vue d'ensemble des composants

```
Pod              ← unité minimale : 1 conteneur (+ éventuellement un sidecar)
  ↑
Deployment       ← maintient N Pods, gère les updates et rollbacks
  ↑
Service          ← adresse stable vers les Pods
  ↑
Ingress          ← point d'entrée depuis l'extérieur

+ ConfigMap      ← configuration non sensible
+ Secret         ← mots de passe, tokens
+ StatefulSet    ← comme Deployment, mais pour les workloads avec état (DB)
```

9 - Namespaces

Les namespaces permettent d’isoler les environnements dans le même cluster

```
kubectl create namespace <namespace> # staging, production etc
..
```

On applique les fichiers YAML en filtrant sur le namespace avec le flag -n :

```
kubectl apply -f configmap.yaml -n <namespace>
```

Chaque namespace a ses propres ressources :

- ses ConfigMaps
- ses Secrets
- ses Deployments.
- …

Les ressources d'un namespace ne voient pas celles d'un autre.

2 - Vue d'ensemble des composants

```
Pod              ← unité minimale : 1 conteneur (+ éventuellement un sidecar)
  ↑
Deployment       ← maintient N Pods, gère les updates et rollbacks
  ↑
Service          ← adresse stable vers les Pods
  ↑
Ingress          ← point d'entrée depuis l'extérieur

+ ConfigMap      ← configuration non sensible
+ Secret         ← mots de passe, tokens
+ StatefulSet    ← comme Deployment, mais pour les workloads avec état (DB)
+ Namespace      ← isolation logique (local, staging, production...)
```

```
Internet
    ▼
[Load Balancer cloud]    ← fourni par DO/OVH en prod
    │                    en local kind : port 80/443 de ta machine
    ▼
[Ingress Controller]     ← ingress-nginx, installé une fois dans le cluster
    │  lit ingress.yaml  ← SEUL point d'entrée externe
    │
    ├── /api/tasks ──▶ [Service task-service] ← service.yaml
    │                       │
    │                  kube-proxy répartit    ← automatique, pas de YAML
    │                  ┌────┴────┐
    │                Pod A     Pod B
    │
    ├── /api/users ──▶ [Service user-service]
    │                  ┌────┴────┐
    │                Pod A     Pod B
    │
    └── /          ──▶ [Service frontend]
                            │
                          Pod A
```

10 - kubectl

Avec Docker, tu dis quoi faire (impératif) :

```
docker run -p 3002:3002 <image>:latest
```

Avec kubectl, tu décris ce que tu veux (déclaratif) :

```
kubectl apply -f deployment.yaml -n local
```

→ Kubernetes lit le YAML, compare avec l'état réel, et agit.

10 - kubectl - Cheat sheet

Crée ou met à jour

```
kubectl apply -f deployment.yaml -n local
```

Liste les pods, services, déploiements

```
kubectl get pods -n local
kubectl get services -n local
kubectl get deployments -n local
```

Liste tout

```
kubectl get all -n local
```

Détail complet + events d'un pod

```
kubectl describe pod <nom> -n local
```

Logs

```
kubectl logs deployment/task-service -n local
```

Exécute un shell dans un pod

```
kubectl exec -it <pod> -n local -- sh
```

10 - kubectl - Gérer les déploiements

Voir la progression d'un déploiement en cours :

```
kubectl rollout status deployment/task-service -n staging
# → "Waiting for deployment: 1 of 2 replicas available"
# → "deployment successfully rolled out"
```

Revenir à une révision précédente :

```
kubectl rollout undo deployment/task-service -n staging
```

Revenir à une révision précise :

```
kubectl rollout undo deployment/task-service -n staging
--to-revision=1
```

10 - kubectl - Gérer les déploiements

Consulter l'historique des révisions :

```
kubectl rollout history deployment/task-service -n staging
# REVISION  CHANGE-CAUSE
# 1         <none>
# 2         <none>
```

Par défaut la colonne `CHANGE-CAUSE` est vide.
On l'enrichit avec `annotate` avant ou après chaque changement :

```
kubectl annotate deployment/task-service -n staging \
  kubernetes.io/change-cause="image v1.1.0 - fix bug login"

kubectl rollout history deployment/task-service -n staging
# REVISION  CHANGE-CAUSE
# 1         image v1.0.0 - version initiale
# 2         image v1.1.0 - fix bug login
```

Fichier YAML	Ressource	Rôle
`deployment.yaml`	Deployment	Maintien N Pods, gère les updates
`service.yaml`	Service	Adresse stable vers les Pods
`configmap.yaml`	ConfigMap	Variables de config injectées
`statefulset.yaml`	StatefulSet	Workload avec état (DB)
`ingress.yaml`	Ingress	Règles de routage HTTP
`secret.yaml`	Secret	Données sensibles injectées (ne pas commiter)
Pas de YAML	kube-proxy	Load balancing interne automatique

Ressources utiles

Kubernetes — concepts

- Kubernetes Documentation — la référence officielle, particulièrement la section Concepts
- Kubernetes Patterns — Bilgin Ibryam & Roland Huß — les patterns d'architecture Kubernetes (sidecar, ambassador, adapter...)

kubectl

- kubectl Cheat Sheet — toutes les commandes, officielle
- kubectl Quick Reference — idem, format condensé

Pratique

- killer.sh — simulateur d'examen CKA, excellent pour pratiquer kubectl
- Play with Kubernetes — cluster Kubernetes dans le navigateur, sans installation